Scheduling Concurrent Coding Agents on Unified-Memory Hardware

15-418/618 Fall 2026 Project Proposal

Jerry Chen (chinjuic) Ethan Chiu (ethanchi)

November 9, 2026

URL

Project page: `https://kogby.github.io/pp-final-project/`
Code: `https://github.com/kogby/pp-final-project`

Summary

We will parallelize the sequential loop of a coding agent (LLM generation → tool execution → ⋯) on two unified-memory machines (NVIDIA DGX Spark & M5 Mackbook Pro). We work at two levels: (1) within one agent, we overlap CPU-side tool execution with GPU-side generation through pipelining and through speculation on the next tool call or its result; (2) across agents, we run several independent agent sessions on one machine with a shared, batched decode stream and concurrent tools. We expect most of the benefit coming from the second level with an estimation from public traces. We plan to replay real agent traces on `llama.cpp` and run tool calls as real programs (e.g., `pytest`, `pandas`). Our central questions are (1) how much parallelization gains on a machine where the extra work shares memory bandwidth and capacity with the critical path, and (2) how much a scheduler gains when it knows each tool's and agent's workload.

Background

The sequential agent loop. A coding agent (Claude Code, Codex, ...) runs in a loop. the LLM generates a response that ends in one or more tool calls, then the harness executes the tools, whose output is appended to the context and the LLM generates again. Each LLM step depends on the previous tool result, and each tool call depends on the previous LLM step, so one request is a long dependency chain.

Public traces of real coding-agent sessions (e.g. TraceLab [1]: 357k LLM rounds, 433k tool calls from 43 users) show roughly 8.8 LLM steps and 10.8 tool calls per user request. Tool latency is heavy-tailed that about 4% of calls that run over a minute account for about 85% of tool time.

Four potential ways to speed up one agent. (Figure 1):

- Pipelining (Conveyor [6]): start executing a tool call as soon as its arguments have been decoded,while the LLM is still generating the rest of its response.

- Speculation (PASTE [5]): predict the next tool call from the pattern of previous calls and execute it during generation. (discard result if no match)
- DAG decomposition (LLMCompiler [7]): execute independent tool calls concurrently.
- Result speculation (value prediction [8]): while a tool runs, fork k generation branches, each conditioned on a likely tool outcome (e.g., build succeeds / fails). When the tool finishes, only keep the branch with correct assumed value.

PASTE and Conveyor hide tool or generation latency behind each other by speculation. All four ways above trade extra work for a potential shorter excution time. Prior systems evaluate on discrete GPUs, while extra work on CPU and DRAM are basically considered free. But that's not the case in a unified memory system.

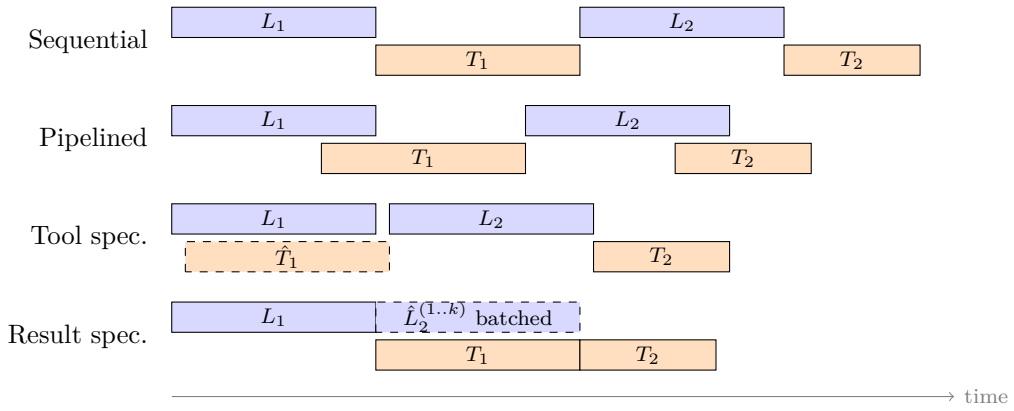


Figure 1: One agent under four schedules. The L lane is decode and the T lane is tool execution.

Estimate how much speedup one agent can gain. At step i, overlap can hide at most $\min(L_i, T_i)$, where L_i is the LLM time and T_i the tool time, so the possible headroom of a request and the best speedup are

$$H = \frac{\sum_i \min(L_i, T_i)}{\sum_i (L_i + T_i)}, \qquad \text{speedup} \le \frac{1}{1 - H}.$$

We computed H over 82k TraceLab requests, with L_i from recorded cloud times or local times measured on the M5 Pro(Table 1). We found that tool time concentrates in a few long calls and 40% of requests call no tool at all. With one-step overlap, a single agent gains at most 1.05–1.07×, and up to 1.20× if speculation could look arbitrarily far ahead. Overlap inside one agent has a ceiling.

LLM time	Context	Tool share	H (one step)	Bound	H (unlimited)	Bound
Cloud (recorded)	–	32%	6.5%	1.07×	16.8%	1.20×
Qwen3-8B	16k	23%	5.9%	1.06×	15.0%	1.18×
Qwen3-8B	32k	17%	5.2%	1.06×	12.6%	1.14×
Qwen3-8B	64k	12%	4.4%	1.05×	9.7%	1.11×

Table 1: Single-agent headroom over TraceLab. "Unlimited" assumes perfect speculation with no lookahead limit.

Many agents on one machine. With the limit indicated above, we then looked into running multiple agents on one machine. When several agents run at once, there are no data dependencies across sessions, which opens up opportunities for parallelism (Figure 2). The GPU decodes every agent that is currently generating as one batch. When fewer than N agents are generating, the freed up batch slots can also run speculative work for agents that are waiting on a tool.

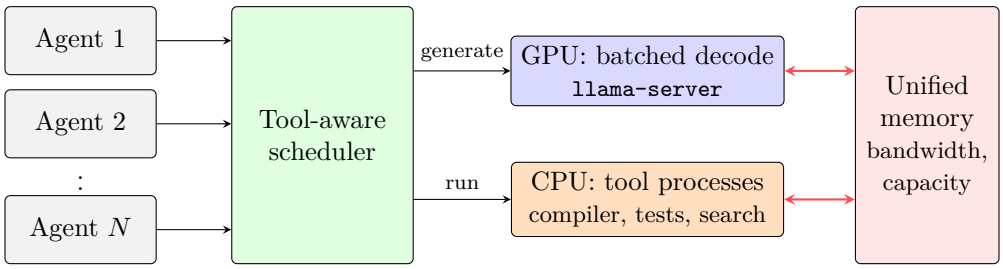


Figure 2: System overview. N independent agents share one machine: the GPU decodes every generating agent in one batch and the CPU runs their tools. Both draw on the same memory, so the scheduler decides when each tool call runs.

Related systems. EdgeAgent [2] schedules several agents on unified memory but simulates tool calls as pauses. it does not measure how generation affects the tools. SERENO [3] studies bandwidth contention between LLM inference and foreground apps on phones. We run real tools and batched generation locally on the unified memory.

The Challenge

What we hope to learn: How many agents can one unified-memory machine run before memory contention stops the gain? When does parallelizing the agent loop pay off on a machine where the parallel work and the critical path share memory, and what does a scheduler need to know about each tool to make that decision?

Dependencies. Within one agent, the workload is a chain by construction. The LLM must see the tool output before it can decide the next call. The only parallelism available is speculative (we guess the next call) or partial (we start the call before the LLM has finished talking), and both have a cost when wrong. The dependency structure is also data-dependent and only revealed at run time, so static scheduling is impossible. The runtime has to decide whether to speculate and how much. A wrong speculation must leave no trace, but tools like compilers, test runners, and `pandas` scripts write files. Solving it with isolation has its own setup cost, which must stay below the latency it hides.

Memory access and communication. On a unified-memory system, the CPU and GPU share one memory pool. Decoding is bandwidth-bound, since every step streams the weights and every batched agent's KV cache through that memory, and many tools also consume significant bandwidth or cache. On a discrete GPU the same tool traffic goes to the host's own DRAM and never touches the GPU's memory. A running tool therefore slows decoding for the whole batch, and decoding slows every running tool. We did preliminary experiments and found that this relation isn't symmetrically. A basic mitigation is to stagger decode away from tools, but that leaves the GPU idle and costs throughput.

Different tools have different footprints. A good policy has to know, per tool, how much it hurts decoding and how much decoding hurts it. We didn't find prior work reports it for agent tools on unified memory.

Agents also interfere with each other. Batching N agents into one decode stream shares the weight reads, but each agent adds its own KV cache to every step. With a long agent context, one agent's KV cache can be as large as the quantized weights, and each added agent slows the step for all the others. Prefill adds a second kind of interference. When a tool returns, its output (for example, a long test or build log) is appended to that agent's context and must be prefilled before the agent can decode again. `llama-server` puts this prefill in the same batch as the other agents' decode steps, so one agent's long tool output slows everyone's generation. If an agent's KV cache is evicted, its whole context has to be prefilled again.

Divergence. Speculative branches behave like divergent lanes. The k branches in a batch generate different lengths, so the batch runs until its longest branch finishes or is cancelled. Mispredicted branches are wasted lanes, and a mispredicted tool speculation is wasted CPU work that still occupies the memory system.

Resources

Hardware.

- NVIDIA GB10 / DGX Spark (128 GB unified memory, 273 GB/s bandwidth, 10P+10E Arm cores, Blackwell GPU, CUDA).
- Apple M5 Pro laptop (24 GB unified memory, 5P+10E CPU cores, 16-core GPU, Metal).
  We plan to develop and iterate on M5 Pro laptop, then do several benchmarks on DGX Spark.

Software. `llama.cpp` / `llama-server` for inference on both CUDA and Metal. We will write the runtime, simulator, and scheduler in Python. Tools are unmodified programs such as `ripgrep`, `clang`/`gcc`, `pytest`, or `pandas`. For profiling we use tools such as Nsight Systems and `perf` on Linux, and `time` and Instruments on macOS.

Datasets. TraceLab [1] (public). We may look for more traces if needed.

Starting point. We start from scratch for the runtime, simulator, and scheduler, and reuse `llama.cpp`.

Goals and Deliverables

Baselines.

Solo One agent on the machine: decode and tool call run in order, while independent calls of one step run concurrently. This serves as reference for scaling efficiency.

Naive N agents share one `llama-server` with continuous batching. Every tool runs when issued. This is the baseline we must beat.

Stagger A naive mitigation of tool slowdown by adding decoding pauses whenever a tool runs.

Ideal We measure each decode step and tool alone, then add up these isolated times along each of the N agents' traces to compute the best achievable time, as if there were no contention between agents and no cost of memory sharing.

Metrics.

- Throughput (agent requests completed per unit time) and per-request active time (time from user prompt to final response), as functions of N.
- Scaling efficiency: throughput(N) / ($N\times$ throughput(1)).
- Interference: slowdown of each tool call and of decoding relative to isolated runs.
- Recovered gap: (Naive − Our time) / (Naive − Ideal), for throughput and active time.

Plan to achieve.

1. **Workload model** from traces: per-agent tool share, tool duration by category, token counts per step, and the single-agent headroom H. We completed M5 Pro (Table 1), and we will repeat the analysis on GB10.
2. **Tool data cards**: a profile of each tool on both machines at different numbers of agents, including metrics like: cores, peak memory, and bandwidth used, how much it slows decoding, and how much decoding slows it.
3. **Multi-agent replay runtime**: build the program that replays N recorded agent sessions at once, with real tools and the same token counts as the original sessions, so every scheduling policy is tested on identical, repeatable workloads.
4. **Scaling study**: sweep N on both platforms for Naive and Stagger and compare against Ideal. Find the bottleneck that stops scaling on each machine, whether it is capacity or bandwidth.
5. **Tool-aware scheduler**: use what we learn about each tool to schedule when tools and generation should run, so that concurrent agents interfere less, and compare it against Naive, Stagger, and Ideal.

Targets.

- Recover at least 50% of the Naive–Ideal gap in p95 active time on the GB10 when the number of agents is greater than a certain number (e.g. $N = 8$).
- Match Stagger's p95 active time with at least 20% higher throughput. Our scheduler should leave the GPU idle less often than Stagger by pausing decoding only for heavy tools to avoid contention, while achieving the same latency for each request.

Hope to achieve.

1. **Speculation in the batch slack**. When fewer agents are generating, the free batch slots can run speculative work for agents that are waiting on a tool. A branch can be committed only if its assumed output is exactly the text the harness would append, so common outcomes must be canonicalized, and speculative tools must run in an isolated copy of the workspace. The accuracy at which speculation breaks even should depend on load.
2. **An online version of the policy**. Sense contention from decode latency instead of relying on data cards.

If progress is slower:

1. Workload model, tool data cards, the replay runtime, the simulator, and the N sweep for Naive and Stagger on both platforms. The deliverable is then the characterization: where scaling stops on each machine and why.
2. If replaying real tools proves unreliable, we replace them with synthetic tools (sleep, CPU loop, bandwidth kernel) calibrated to the data cards.

Poster. We plan to includes plot such as: throughput and p95 active time versus the number of agent on both platforms, with the baselines and our scheduling policy measured against Ideal. We will also show a timeline of N replayed agents under different scheduling along with live replays on the laptop.

Platform Choice

Our question is about machines whose CPU and GPU share memory, so we use two unified-memory machines that cover both vendors and operating systems that ship them. They differ by about $5\times$ in memory capacity, so we expect the M5 Pro laptop to run out of KV-cache space first and the GB10 to saturate bandwidth first. We use `llama.cpp` because it runs on both CUDA and Metal with the same quantized weights, which removes the engine as a variable.

Development order. We develop and debug on the M5 Pro laptop and use the GB10 mainly as the measurement machine. We check correctness under every policy: a replayed agent should produce the same tool outputs, the same token counts per step, and the same final transcript whether it runs alone or with $N - 1$ others. Only timing may differ between runs and platforms.

Schedule

Dates	**Goal**
Nov 9–12	Build the workload model from traces and measure single-agent headroom (done on laptop already). Measure prefill and decode rates versus context length and batch size.
Nov 13–15	Profile how each tool uses cores, memory, and bandwidth, alone and concurrent with batched generation.
Nov 16–19	Build the multi-agent runtime and check that every agent's transcript is identical alone and in a group.
Nov 20–22	Build the ideal baseline and validate it at $N = 1$. Write the milestone report.
Nov 23	**Milestone report due.**
Nov 23–26	Profile the tools on the GB10. Sweep N with naive concurrency on both machines.
Nov 27–29	Add stagger baseline. Measure the gap to the simulator on each machine and profile it.
Nov 30–Dec 2	Build and refine the policy.
Dec 3–5	Add the limit on how many agents could run at once and to place core placement on the GB10. Compare the policy against Naive, Stagger, and Ideal.
Dec 6–8	Buffer time or time for extra goals.
Dec 9–10	Write the final report.
Dec 10	**Final report due.**
Dec 11	**Poster session.**

References

[1] Zhu et al. TraceLab: Characterizing Coding Agent Workloads for LLM Serving. arXiv:2606.30560, 2026. `https://github.com/uw-syfi/TraceLab`

[2] Y. Long, Y. Wei, K. Wu, J. Wei, D. Huang, J. Du. EdgeAgent: Orchestrating On-Device LLM Inference for End-User Multi-Agent Systems on CPU-GPU Unified Memory Architectures. ASPLOS 2027.

[3] T. Xin, X. Shi, M. Dong, Z. Mi. Inference in the Shadows: Taming Memory Bandwidth Contention in Mobile LLM Inference with SERENO. OSDI 2026.

[4] Mars et al. Bubble-Up: Increasing Utilization in Modern Warehouse Scale Computers via Sensible Co-locations. MICRO 2011.

[5] Y. Sui, H. Zhao, R. Ma, Z. He, H. Wang, J. Li, Y. Yang. Act While Thinking: Accelerating LLM Agents via Pattern-Aware Speculative Tool Execution. arXiv:2603.18897, 2026.

[6] Y. Xu, X. Kong, T. Chen, D. Zhuo. Conveyor: Efficient Tool-aware LLM Serving with Tool Partial Execution. arXiv:2406.00059, 2024.

[7] Kim et al. An LLM Compiler for Parallel Function Calling. ICML 2024.

[8] M. H. Lipasti, C. B. Wilkerson, J. P. Shen. Value Locality and Load Value Prediction. ASPLOS 1996.